Chapter 42 Using EMS memory

This chapter describes how to use the Hunter 16 expanded memory system (EMS) device driver which is available from your dealer.

42.1 The facilities

Hunter 16 RAM can be organised into three separate areas:

- Main (DOS) memory
- RAM disk (drive C)
- Expansion memory (EMS)

The RAM is configured into these three areas by using the UTIL utility, as described in Part 2. You must have main memory (to execute programs) and a RAM disk (for file storage). You can only configure EMS memory if expansion memory (beyond 1MByte) has been fitted to the unit.

To use it, you must:

1. Fit expansion memory to **Hunter 16** (ie memory above 1MByte).
2. Use UTIL to configure some of this memory for use as EMS. Remember that this will erase all user files from the fixed disk, so archive them first, then copy them back afterwards!
3. Install the EMS device driver (EMSDRIVE.SYS) by copying it onto the fixed disk.
4. Add the line DEVICE=EMSDRIVE .SYS to the CONFIG.SYS file on the boot disk root directory (usually drive C). (There are no parameters in the device driver line in CONFIG.SYS because these parameters are provided in step 2.)
5. Reboot **Hunter 16**.

EMS memory provides a standard way of expanding the memory map of the 8086 processor, by paging up to four 16 KByte blocks of memory at a time into a 'spare' 64 KByte segment of the 1MByte addressing range, as required. It can be used to enhance the performance of certain major applications (instead of temporary files), or by a range of simpler utilities (such as disk cache and print spooler routines).

The Hunter 16 expanded memory manager (EMM) fully complies with v3.20 of the Lotus/Intel/Microsoft specification for expanded memory. If you simply want to use utilities or applications that are written for this standard, or already know how to program for it, you may safely ignore the rest of this chapter.

The main EMS facilities are accessed as INT 67H functions:

Access by:	**Uses:**
Normal code	Functions 40H..46H. Get status, page frame address and unallocated page count. Allocate, map and de-allocate pages. Get EMM version.
TSR program	Functions 47H and 48H. Save and restore map.
Device driver **INT service routine** **EMS utility**	Functions 4BH..4DH. Get handle and page allocation data.
Multitasking O/S	Function 4EH. Get or set page map.

For brief details, see Chapter 30. For full details of the EMM, see *The Lotus/Intel/Microsoft Expanded Memory Specification*, version 3.20.

42.2 Using the facilities

To use EMS memory a 'normal' **Hunter 16** program (ie one which is not a TSR, device driver or interrupt service routine) would:

- Check that the EMM is installed (function 40H).
- Get the page frame base address (function 41H).
- Check how many pages are available (function 42H).
- Request the pages required and remember the handle returned (function 43H).
- Within the main program loop:
    - Select pages to access, by using map requests (function 44H).
    - Access data in the selected pages like conventional memory.
- When finished, close the EMM (function 45H), to free the pages for other uses.

However, if you are writing a program solely for use on **Hunter 16**, the EMM page frame base address is always C0000H, so you may safely omit the second step.

For example, suppose you need to access (and update) a table of 128 KByte of data which is stored on disk, and there is not enough room in main memory. If there is sufficient EMS memory, you could read the data into EMS memory, for faster access; if not, you must access the disk each time. You could implement code to do this as follows.

For simplicity, this code maps each logical page to the same physical page each time: logical page 0 or 4 to physical page 0 (C0000..C3FFFH), page 1 or 5 to 1, page 2 or 6 to 2 and 3 or 7 to 3.

```
;****************************
;* EXAMPLE: USING EMS MEMORY *
;****************************

START:
check if EMM installed (INT 67H, 40OH)

if EMM is installed
set PAGES NEEDED to 8              ;128K = 8* 16K PAGES
     get unallocated pages (INT 67H, 42H)

if count < PAGES NEEDED            ;NOT ENOUGH EMS
or EMM is not instal led           ;NO EMS
     reset EMSFLAG                 ;SO REMEMBER TO USE DISK
     call OPENDATA                 ;AND PREPARE TO USE DISK

else                               ;ENOUGH EMS, SO USE IT

     ;SET UP EMS PAGES
     set EMSFLAG
     request PAGES NEEDED (INT 67H, 43)
     store HANDLE returned

     ;READ DATA INTO EMS
     call MAP (0)                  ;MAP IN PAGES 0..3
     ..call MAP(3)
     open input file               ;DATA TO READ IN
     call READ 64                  ;READ 1ST 64K
     call MAP(4)                   ;MAP IN PAGES 4..7
     ..MAP(7)
     call READ 64                  ;READ 2ND 64K
```

```
     ;REMEMEBER PAGES
     set MAPPEDO to 0    ;REMEMBER PAGES 0..3 ARE MAPPED OUT
     ..set MAPPED3 to 0
     set MAPPED4 to 1    ;REMEMBER PAGES 4..7 ARE MAPPED IN
     ..set MAPPED7 to 1

;MAIN PROCESSING LOOP
;
;
;BEFORE ANY EXTERNAL DATA ACCESs
if EMSFLAG                          ;USE EMS
     set n to physical page containing byte
     if MAPPEDn=0 calI MAP(n)       ;GET PAGE IF IT IS MAPPED OUT
     write or read byte             ;ACCESS AS MEMORY
else
     call GETBYTE(z) or PUTBYTE(z) ;NO EMS, SO USE DIRECT DISK
ACCESS
;
;
;END OF MAIN PROCESSING

;TIDY UP

if EMSFLAG                          ;USING EMS MEMORY SO:
     ;REWRITE DATA FILE
     call MAP(0)                    ;MAP IN 1ST 64K OF EMs
     ..call MAP(3)
     position at file start
     call WRITE 64                  ;REWRITE 1ST 64K
     call MAP(4)                    ;MAP IN 2ND 64K OF EMS
     ..call MAP(7)
     call WRITE 64                  ;REWRITE 2ND 64K

     ;RELEASE EMS FOR OTHER USES
     de-allocate pages (INT 67H, 45H)

close file

exit to DOS
```

```
;MAP ROUTINE: USED TO ACCESS AN EMS PAGE

MAP:
set p to (n NOD 4)        ;CALCULATE PHYSICAL PAGE TO USE
map logical page n to physical page p (INT 67H, 44H)
                          ;SELECT THE PAGE
set MAPPEDn to 1          ;MARK NEW PAGE AS MAPPED IN
n = (n + 4) MOD 8         ;CALCULATE DISPLACED PAGE
set MAPPEDn to 0          ;MARK DISPLACED PAGE AS MAPPED
OUT
return
```

For simplicity, the disk access routines used for EMS (READ 64, WRITE 64) have been omitted, as well as the routines which would be used to access the disk data directly, if sufficient EMS memory is not provided (GETBYTE, PUTBYTE, etc).